

RESTful E-Commerce Product Management API

Technical Report — Assignment 1

Course: CCCS-425-764 Web Services

Semester: Summer 2026

McGill University, School of Continuing Studies

Student Name: Brad Cibula

Student ID: 260295088

Due Date: June 28, 2026

Table of Contents

Table of Contents	2
1. Introduction	3
2. System Architecture	3
3. Database Design	4
3.1 Database Creation	4
3.2 Table Creation	5
3.3 Dataset Insertion	5
4. Entity Classes	6
5. Repository Layer	7
6. REST Controller Design	8
6.1 Service Layer	9
7. API Endpoints	10
8. Error Handling Strategy	11
9. Testing Results	13
9.1 Project Setup & Running Application	13
9.2 Create Product (POST)	15
9.3 Retrieve All Products (GET)	16
9.4 Retrieve Product by ID (GET)	17
9.5 Search Products (GET)	18
9.6 Retrieve Products by Category (GET)	19
9.7 Update Product (PUT)	20
9.8 Delete Product (DELETE)	21
9.9 JSON Response Format	22
9.10 XML Response Format	23
9.11 Error Handling Examples	24
10. Conclusion	26
11. AI Transparency and Acknowledgment	26

1. Introduction

This project implements a RESTful Product Management API for SmartShop, an online retail company that requires a backend service to manage its product catalog. The API provides a structured way to create, retrieve, update, search, and delete product records while storing the data in a persistent MySQL database. It is designed using Spring Boot, Spring Data JPA, Hibernate, and Maven, with REST endpoints that support both JSON and XML responses. The system also includes basic error handling so that API clients receive meaningful HTTP status codes when requests succeed, fail validation, or reference products that do not exist.

2. System Architecture

The application follows a standard layered architecture, separating concerns across four layers:

Layer	Responsibility
REST Controller	Handles HTTP requests/responses, routes to service layer
Service	Business logic, including discount calculation
Repository	Data access via Spring Data JPA
Entity / Model	Maps Java objects to database table rows

Architecture flow: Client → REST Controller → Service Layer → Repository Layer → MySQL Database.

3. Database Design

The application uses a MySQL database named **smartshop**, with a single table named **products**. The table schema is automatically generated by Hibernate from the Product entity class (ddl-auto=update).

Column	Type
product_id	BIGINT, Primary Key, Auto Increment
title	VARCHAR(255)
description	VARCHAR(255)
price	DOUBLE
inventory_quantity	INT
image_url	VARCHAR(255)
category	VARCHAR(255)

Note: discountedPrice is a calculated, transient field and is not persisted in the database.

3.1 Database Creation

```
mysql> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| smartshop |
| sys |
+-----+
5 rows in set (0.00 sec)
```

3.2 Table Creation

```
mysql> USE smartshop;
Database changed
mysql> DESCRIBE products;
```

Field	Type	Null	Key	Default	Extra
product_id	bigint	NO	PRI	NULL	auto_increment
category	varchar(255)	YES		NULL	
description	varchar(255)	YES		NULL	
image_url	varchar(255)	YES		NULL	
inventory_quantity	int	YES		NULL	
price	double	NO		NULL	
title	varchar(255)	YES		NULL	

7 rows in set (0.00 sec)

3.3 Dataset Insertion

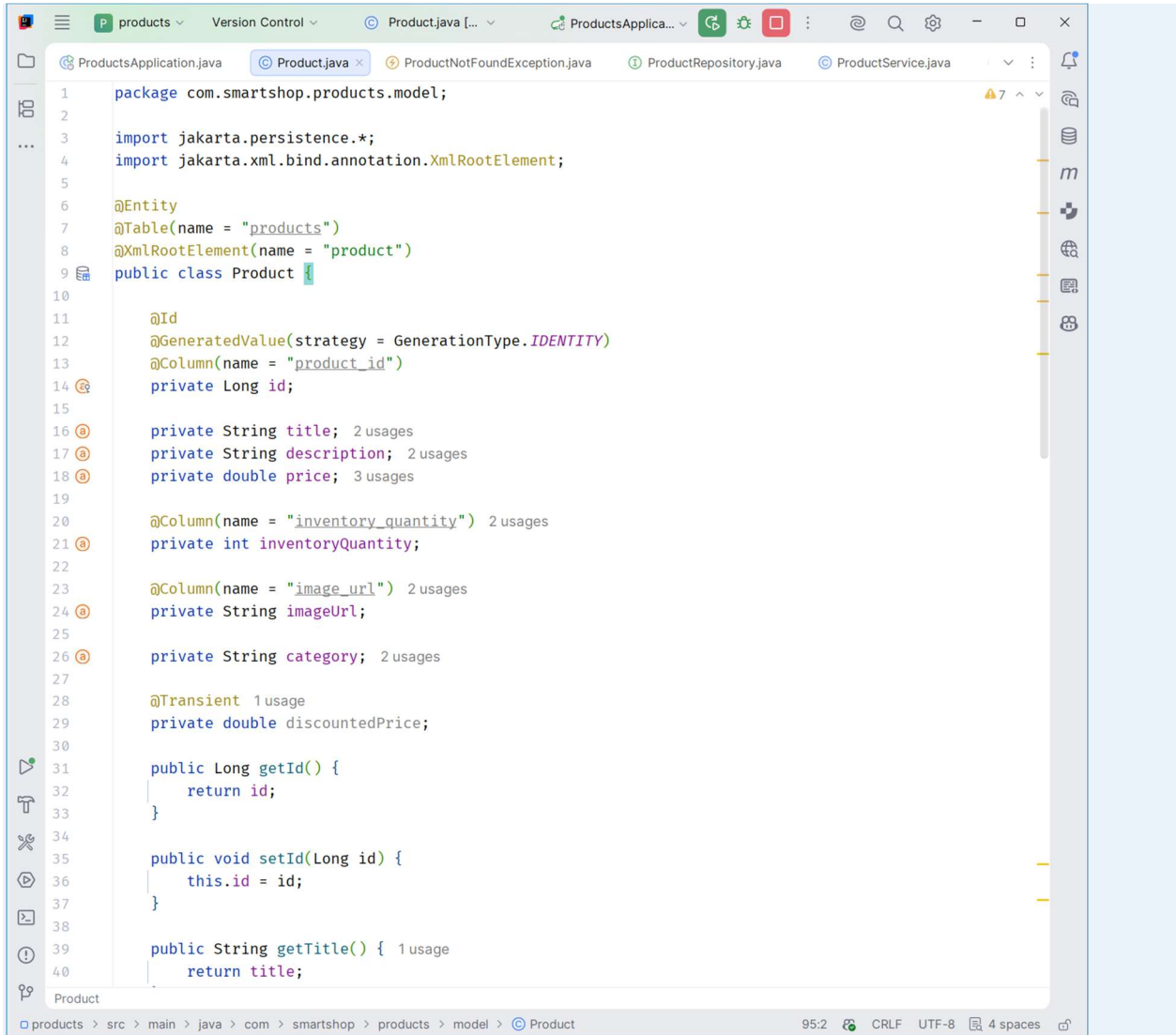
```
mysql> SELECT * FROM products;
```

product_id	category	description	image_url	inventory_quantity	price	title
1	Electronics	Ergonomic wireless mouse	mouse.jpg	120	29.99	Wireless Mouse
2	Electronics	RGB gaming keyboard	keyboard.jpg	45	89.99	Mechanical Keyboard
3	Furniture	Adjustable ergonomic chair	chair.jpg	15	249.99	Office Chair
4	Accessories	Aluminum laptop stand	stand.jpg	80	39.99	Laptop Stand
5	Electronics	Multiport USB-C hub	hub.jpg	60	59.99	USB-C Hub
6	Electronics	4K UHD monitor	monitor.jpg	22	399.99	Monitor 27 Inch
7	Furniture	LED desk lamp	lamp.jpg	75	34.99	Desk Lamp
8	Electronics	HD Full HD webcam	webcam.jpg	50	69.99	Webcam
9	Electronics	Noise-cancelling headphones	headphones.jpg	35	149.99	Headphones
10	Storage	1TB portable SSD	ssd.jpg	40	129.99	Portable SSD

10 rows in set (0.00 sec)

4. Entity Classes

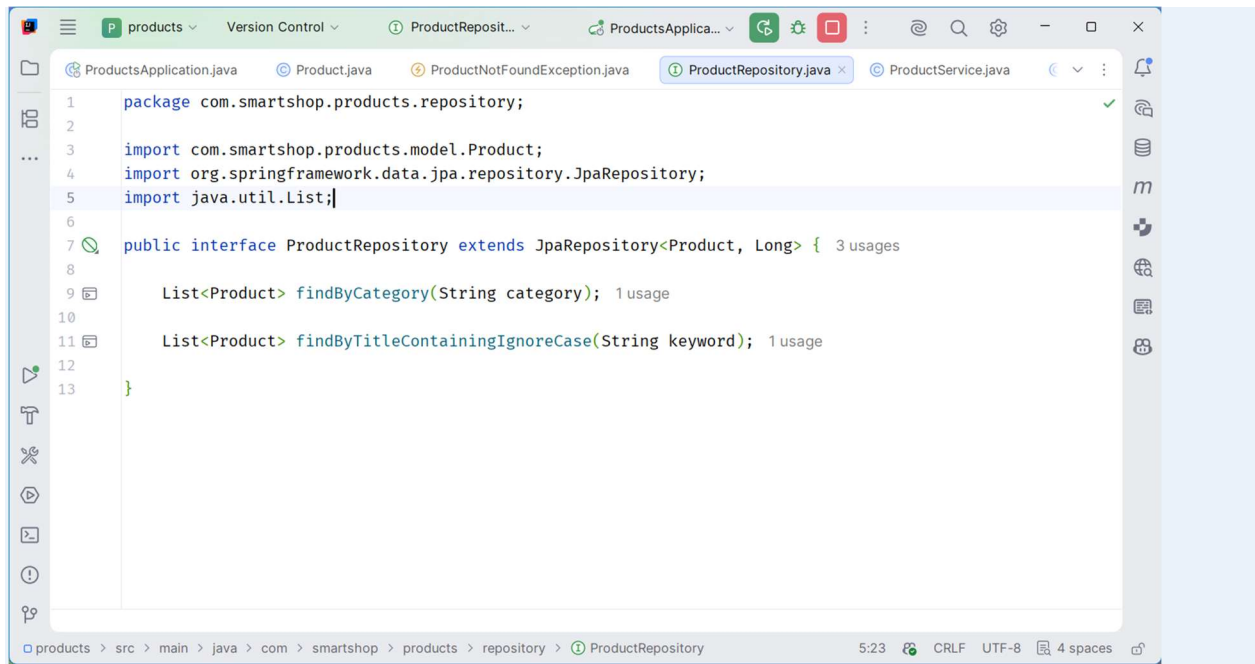
The Product entity class maps directly to the products table using JPA annotations (@Entity, @Table, @Id, @GeneratedValue, @Column, @Transient).



Note: The screenshot shows the main portion of the Product entity class. The complete source code is included with the submitted project files.

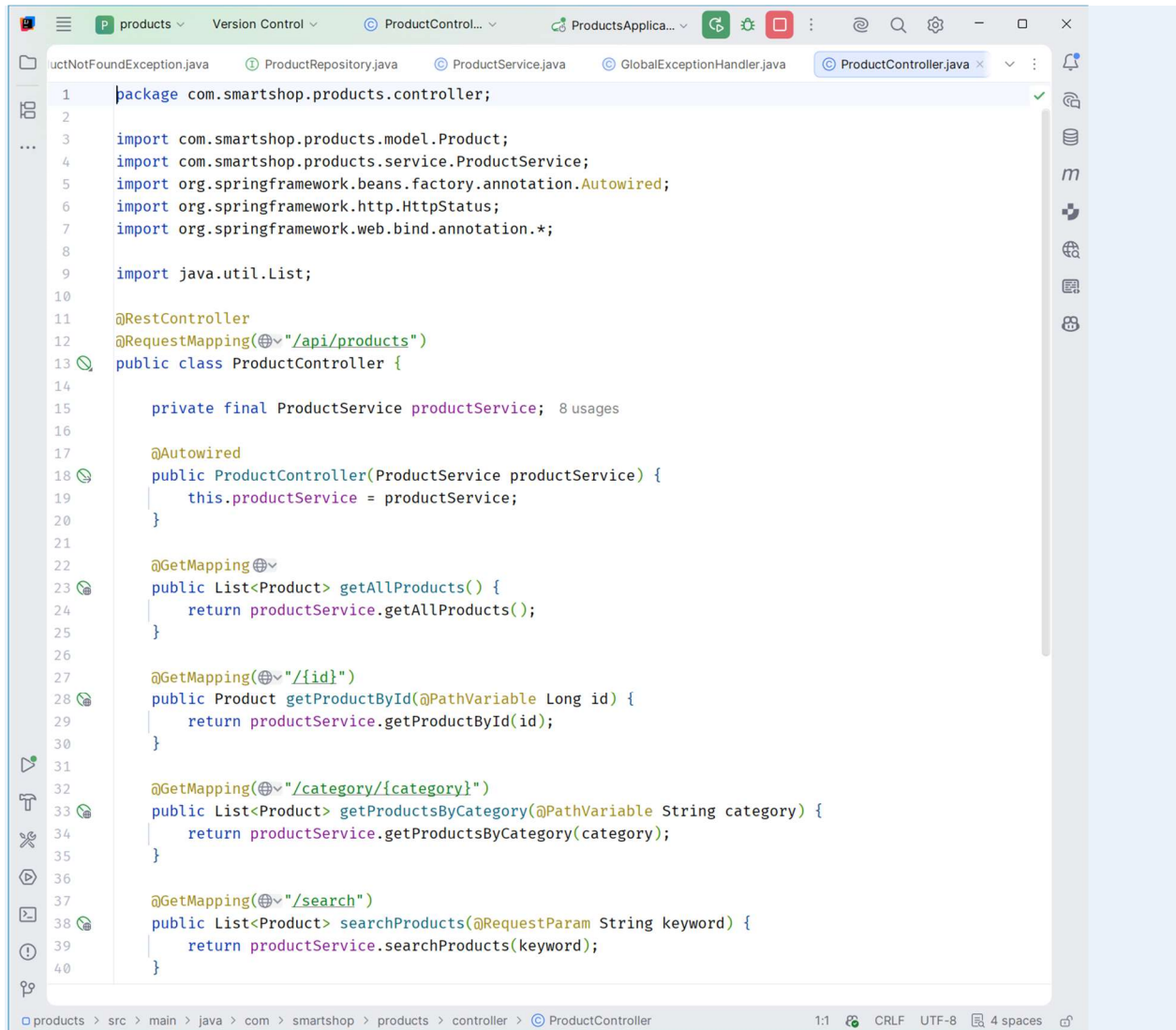
5. Repository Layer

ProductRepository extends Spring Data JPA's JpaRepository, providing built-in CRUD methods (save, findAll, findById, deleteById) along with two custom derived query methods: findByCategory and findByTitleContainingIgnoreCase.



6. REST Controller Design

ProductController exposes RESTful endpoints under /api/products, delegating all business logic to ProductService. HTTP methods are mapped according to standard REST conventions (GET for read, POST for create, PUT for update, DELETE for remove).



```

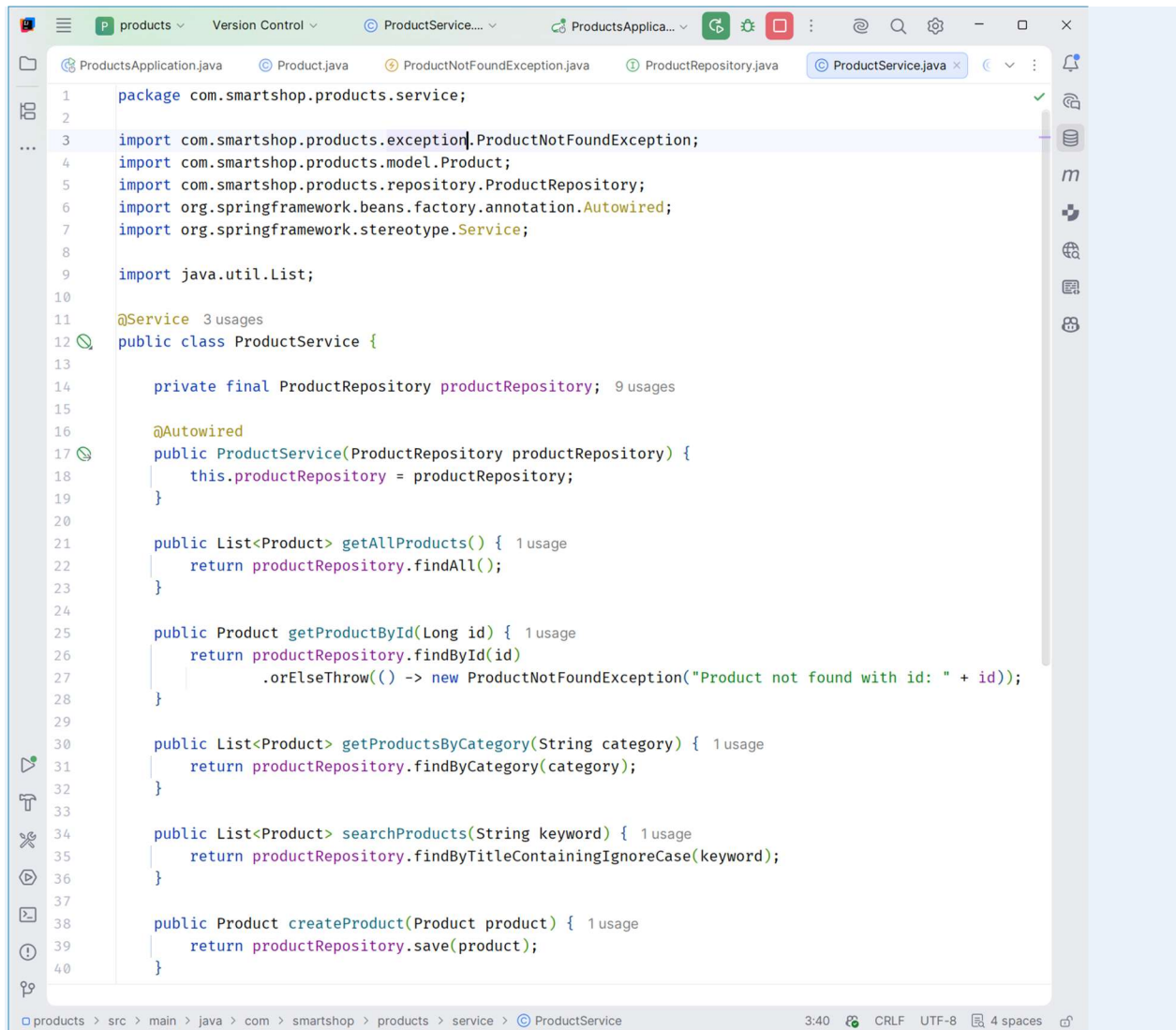
1 package com.smartshop.products.controller;
2
3 import com.smartshop.products.model.Product;
4 import com.smartshop.products.service.ProductService;
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.http.HttpStatus;
7 import org.springframework.web.bind.annotation.*;
8
9 import java.util.List;
10
11 @RestController
12 @RequestMapping("/api/products")
13 public class ProductController {
14
15     private final ProductService productService; 8 usages
16
17     @Autowired
18     public ProductController(ProductService productService) {
19         this.productService = productService;
20     }
21
22     @GetMapping
23     public List<Product> getAllProducts() {
24         return productService.getAllProducts();
25     }
26
27     @GetMapping("/{id}")
28     public Product getProductById(@PathVariable Long id) {
29         return productService.getProductById(id);
30     }
31
32     @GetMapping("/{category}/{category}")
33     public List<Product> getProductsByCategory(@PathVariable String category) {
34         return productService.getProductsByCategory(category);
35     }
36
37     @GetMapping("/search")
38     public List<Product> searchProducts(@RequestParam String keyword) {
39         return productService.searchProducts(keyword);
40     }
41

```

Note: The screenshot shows the main portion of the ProductController class. The complete source code is included with the submitted project files.

6.1 Service Layer

The ProductService class contains the main business logic for the API. It retrieves products through the repository layer, supports create, update, delete, search, and category operations, and throws a ProductNotFoundException when a requested product does not exist.



Note: The screenshot shows the main portion of the class. The complete source code is included with the submitted project files.

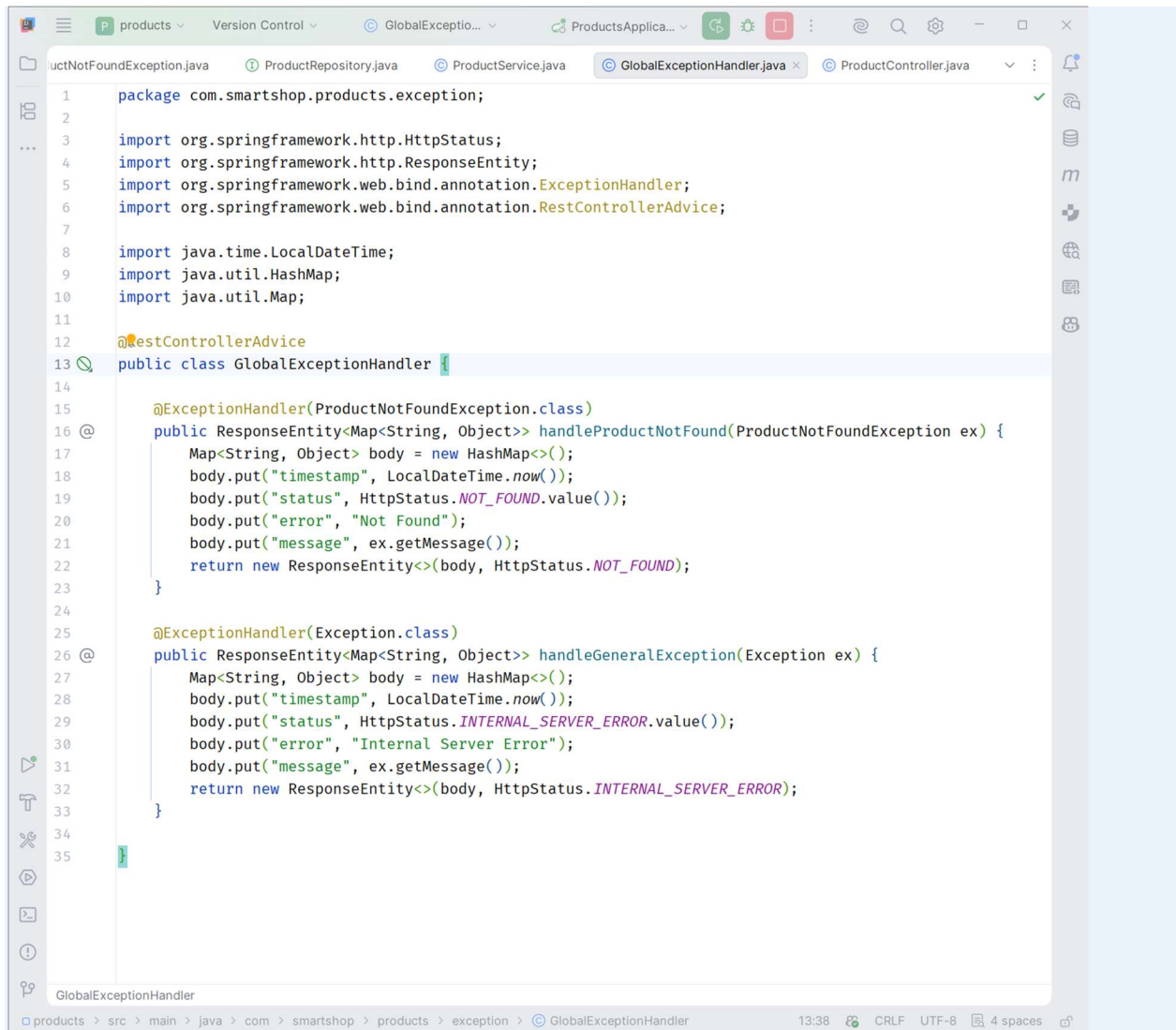
7. API Endpoints

Endpoint	Description
GET /api/products	Retrieve all products
GET /api/products/{id}	Retrieve a product by ID
GET /api/products/category/{category}	Retrieve products by category
GET /api/products/search?keyword=	Search products by keyword
POST /api/products	Create a new product
PUT /api/products/{id}	Update an existing product
DELETE /api/products/{id}	Delete a product

8. Error Handling Strategy

The API uses a `GlobalExceptionHandler` class annotated with `@RestControllerAdvice` to centralize error handling across the application. When a requested product cannot be found, the application throws a `ProductNotFoundException` and returns a 404 Not Found response with a structured error body. The handler also includes a general exception handler for unexpected errors, returning a 500 Internal Server Error response. This approach keeps error handling separate from the controller logic and ensures API clients receive consistent status codes and error messages.

Situation	HTTP Status Code
Product found	200 OK
Product created	201 Created
Invalid request	400 Bad Request
Product not found	404 Not Found
Internal error	500 Internal Server Error



```
1 package com.smartshop.products.exception;
2
3 import org.springframework.http.HttpStatus;
4 import org.springframework.http.ResponseEntity;
5 import org.springframework.web.bind.annotation.ExceptionHandler;
6 import org.springframework.web.bind.annotation.RestControllerAdvice;
7
8 import java.time.LocalDateTime;
9 import java.util.HashMap;
10 import java.util.Map;
11
12 @RestControllerAdvice
13 public class GlobalExceptionHandler {
14
15     @ExceptionHandler(ProductNotFoundException.class)
16     public ResponseEntity<Map<String, Object>> handleProductNotFound(ProductNotFoundException ex) {
17         Map<String, Object> body = new HashMap<>();
18         body.put("timestamp", LocalDateTime.now());
19         body.put("status", HttpStatus.NOT_FOUND.value());
20         body.put("error", "Not Found");
21         body.put("message", ex.getMessage());
22         return new ResponseEntity<>(body, HttpStatus.NOT_FOUND);
23     }
24
25     @ExceptionHandler(Exception.class)
26     public ResponseEntity<Map<String, Object>> handleGeneralException(Exception ex) {
27         Map<String, Object> body = new HashMap<>();
28         body.put("timestamp", LocalDateTime.now());
29         body.put("status", HttpStatus.INTERNAL_SERVER_ERROR.value());
30         body.put("error", "Internal Server Error");
31         body.put("message", ex.getMessage());
32         return new ResponseEntity<>(body, HttpStatus.INTERNAL_SERVER_ERROR);
33     }
34
35 }
```

GlobalExceptionHandler

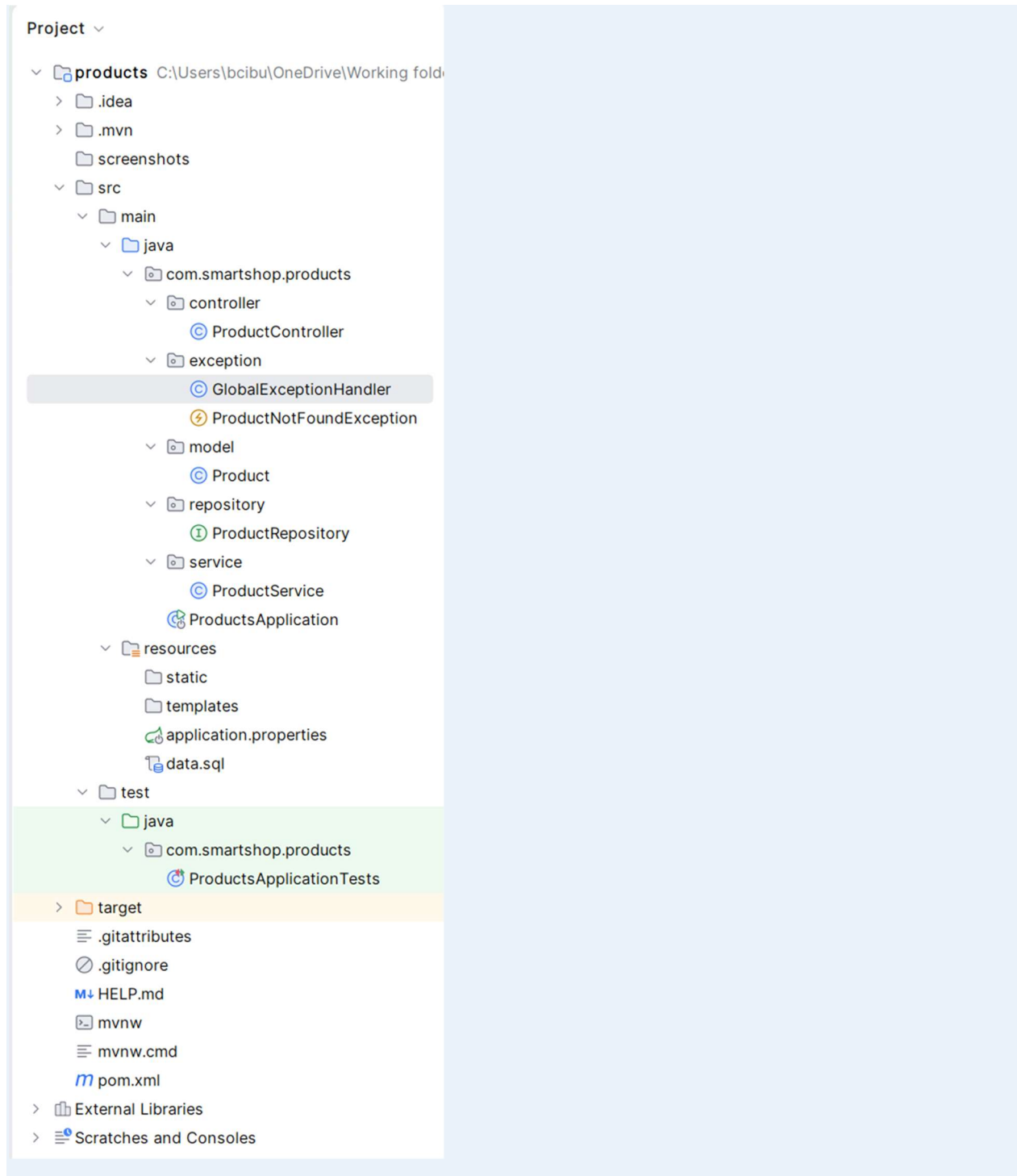
products > src > main > java > com > smartshop > products > exception > GlobalExceptionHandler

13:38 CRLF UTF-8 4 spaces

9. Testing Results

All endpoints were tested using Postman, in both JSON and XML formats, including error scenarios.

9.1 Project Setup & Running Application



The screenshot displays an IDE with two windows. The top window, titled 'application.properties', shows the following configuration:

```
1 spring.application.name=products
2 spring.datasource.url=jdbc:mysql://localhost:3306/smartshop
3 spring.datasource.username=root
4 spring.datasource.password=
5
6 spring.jpa.hibernate.ddl-auto=update
7 spring.jpa.show-sql=true
8
9 spring.sql.init.mode=always
10
11
```

The bottom window, titled 'Run', shows the console output of the application. The output includes the following log messages:

```
2026-06-21T14:50:43.927-04:00 INFO 26620 --- [products] [ restartedMain] c.s.products.ProductsApplication : Starting ProductsApplication using Java 17.0.19 with PID 26620 (C:\Users\bcibu...
2026-06-21T14:50:43.930-04:00 INFO 26620 --- [products] [ restartedMain] c.s.products.ProductsApplication : No active profile set, falling back to 1 default profile: "default"
2026-06-21T14:50:43.959-04:00 INFO 26620 --- [products] [ restartedMain] e.DevToolsPropertyDefaultsPostProcessor : Devtools property defaults active! Set 'spring.devtools.add-properties' to 'fa
2026-06-21T14:50:44.385-04:00 INFO 26620 --- [products] [ restartedMain] s.d.f.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in DEFAULT mode.
2026-06-21T14:50:44.421-04:00 INFO 26620 --- [products] [ restartedMain] s.d.f.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in 29 ms. Found 1 JPA repository inte
2026-06-21T14:50:44.750-04:00 INFO 26620 --- [products] [ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2026-06-21T14:50:44.750-04:00 INFO 26620 --- [products] [ restartedMain] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-06-21T14:50:44.759-04:00 INFO 26620 --- [products] [ restartedMain] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.55]
2026-06-21T14:50:44.788-04:00 INFO 26620 --- [products] [ restartedMain] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2026-06-21T14:50:44.849-04:00 INFO 26620 --- [products] [ restartedMain] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 828 ms
2026-06-21T14:50:44.849-04:00 INFO 26620 --- [products] [ restartedMain] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
2026-06-21T14:50:45.058-04:00 INFO 26620 --- [products] [ restartedMain] com.zaxxer.hikari.HikariPool : HikariPool-1 - Added connection com.mysql.cj.jdbc.ConnectionImpl@575c85a7
2026-06-21T14:50:45.059-04:00 INFO 26620 --- [products] [ restartedMain] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
2026-06-21T14:50:45.139-04:00 INFO 26620 --- [products] [ restartedMain] o.hibernate.jpa.internal.util.LogHelper : HH0000204: Processing PersistenceUnitInfo [name: default]
2026-06-21T14:50:45.170-04:00 INFO 26620 --- [products] [ restartedMain] org.hibernate.Version : HH0000412: Hibernate ORM core version 6.6.53.Final
2026-06-21T14:50:45.195-04:00 INFO 26620 --- [products] [ restartedMain] o.h.c.internal.RegionFactoryInitiator : HH0000026: Second-level cache disabled
2026-06-21T14:50:45.383-04:00 INFO 26620 --- [products] [ restartedMain] o.s.o.j.p.SpringPersistenceUnitInfo : No LoadTimeWeaver setup; ignoring JPA class transformer
2026-06-21T14:50:45.448-04:00 INFO 26620 --- [products] [ restartedMain] org.hibernate.orm.connections.pooling : HH010001005: Database info:
Database JDBC URL [Connecting through datasource 'HikariDataSource (HikariPool-1)']
Database driver: undefined/unknown
Database version: 8.0.46
Autocommit mode: undefined/unknown
Isolation level: undefined/unknown
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown
2026-06-21T14:50:45.943-04:00 INFO 26620 --- [products] [ restartedMain] o.h.e.t.j.p.i.JtaPlatformInitiator : HH0000409: No JTA platform available (set 'hibernate.transaction.jta.platform'
2026-06-21T14:50:45.976-04:00 INFO 26620 --- [products] [ restartedMain] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-06-21T14:50:46.179-04:00 WARN 26620 --- [products] [ restartedMain] jpabase.Configuration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may
2026-06-21T14:50:46.431-04:00 INFO 26620 --- [products] [ restartedMain] o.s.b.d.a.OptionalLiveReloadServer : LiveReload server is running on port 35729
2026-06-21T14:50:46.460-04:00 INFO 26620 --- [products] [ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2026-06-21T14:50:46.468-04:00 INFO 26620 --- [products] [ restartedMain] c.s.products.ProductsApplication : Started ProductsApplication in 2.827 seconds (process running for 3.574)
```

9.2 Create Product (POST)

The screenshot displays a REST client interface for the SmartShop Product Management API. The active tab is 'POST Create Product' with the endpoint `{{baseUrl}}/api/products`. The request body is a JSON object representing a product:

```
1 {
2   "title": "Gaming Mousepad",
3   "description": "Extended RGB mousepad",
4   "price": 24.99,
5   "inventoryQuantity": 100,
6   "imageUrl": "mousepad.jpg",
7   "category": "Accessories"
8 }
```

The response status is **201 Created** with a response time of 93 ms and a body size of 355 B. The response body is a JSON object:

```
1 {
2   "id": 11,
3   "title": "Gaming Mousepad",
4   "description": "Extended RGB mousepad",
5   "price": 24.99,
6   "inventoryQuantity": 100,
7   "imageUrl": "mousepad.jpg",
8   "category": "Accessories",
9   "discountedPrice": 22.49
10 }
```

The interface includes tabs for Docs, Params, Authorization, Headers, Body, Scripts, Tests, and Settings. The Body tab is selected, showing the request and response JSON. The bottom status bar indicates 'Connect Git', 'Console', 'Terminal', and 'Globals Vault Tools'.

9.3 Retrieve All Products (GET)

The screenshot displays a REST client interface for the SmartShop Product API. The request is a GET to `{{baseUrl}}/api/products`, which has been successfully executed, returning a `200 OK` status with a response time of 17 ms and a body size of 1.9 KB.

The response is shown in JSON format, containing an array of three product objects:

```
1 [
2   {
3     "id": 1,
4     "title": "Wireless Mouse",
5     "description": "Ergonomic wireless mouse",
6     "price": 29.99,
7     "inventoryQuantity": 120,
8     "imageUrl": "mouse.jpg",
9     "category": "Electronics",
10    "discountedPrice": 26.99
11  },
12  {
13    "id": 2,
14    "title": "Mechanical Keyboard",
15    "description": "RGB gaming keyboard",
16    "price": 89.99,
17    "inventoryQuantity": 45,
18    "imageUrl": "keyboard.jpg",
19    "category": "Electronics",
20    "discountedPrice": 80.99
21  },
22  {
23    "id": 3,
24    "title": "Office Chair",
25    "description": "Adjustable ergonomic chair",
26    "price": 249.99,
27    "inventoryQuantity": 15,
28    "imageUrl": "chair.jpg",
29  }
30 ]
```


9.4 Retrieve Product by ID (GET)

The screenshot displays a REST client interface for the SmartShop Product Management API. The active endpoint is **GET Get Product By ID** with the URL `{{baseUrl}}/api/products/1`. The request method is **GET**. The response status is **200 OK** with a response time of 26 ms and a body size of 348 B.

The **Query Params** section is empty.

The **Body** section shows the JSON response:

```
1 {
2   "id": 1,
3   "title": "Wireless Mouse",
4   "description": "Ergonomic wireless mouse",
5   "price": 29.99,
6   "inventoryQuantity": 120,
7   "imageUrl": "mouse.jpg",
8   "category": "Electronics",
9   "discountedPrice": 26.99
10 }
```

The interface includes tabs for Docs, Params, Authorization, Headers, Body, Scripts, Tests, and Settings. The bottom status bar shows "Connect Git", "Console", "Terminal", and "Globals Vault Tools".

9.5 Search Products (GET)

The screenshot displays a REST client interface with the following components:

- Header Bar:** Includes navigation icons, a search bar, and buttons for 'Invite', 'Upgrade', and user profile.
- Tab Bar:** Shows a list of API endpoints, with 'GET Search P' selected.
- Request Bar:** The method is 'GET' and the URL is '({baseUrl})/api/products/search?keyword=mouse'. A 'Send' button is on the right.
- Params Tab:** A table for query parameters is shown below the request bar.

Key	Value	Description	Bulk Edit
✓ keyword	mouse		
Key	Value	Description	
- Body Tab:** The response is displayed in JSON format. The status is '200 OK' with a response time of '22 ms' and a size of '350 B'.

```
1 [
2   {
3     "id": 1,
4     "title": "Wireless Mouse",
5     "description": "Ergonomic wireless mouse",
6     "price": 29.99,
7     "inventoryQuantity": 120,
8     "imageUrl": "mouse.jpg",
9     "category": "Electronics",
10    "discountedPrice": 26.99
11  }
12 ]
```
- Footer:** Includes 'Connect Git', 'Console', 'Terminal', and status indicators.

9.6 Retrieve Products by Category (GET)

The screenshot displays a REST client interface with the following details:

- URL:** `{{baseUrl}}/api/products/category/Electronics`
- Method:** GET
- Response Status:** 200 OK, 35 ms, 1.22 KB
- Response Body (JSON):**

```
1 [
2   {
3     "id": 1,
4     "title": "Wireless Mouse",
5     "description": "Ergonomic wireless mouse",
6     "price": 29.99,
7     "inventoryQuantity": 120,
8     "imageUrl": "mouse.jpg",
9     "category": "Electronics",
10    "discountedPrice": 26.99
11  },
12  {
13    "id": 2,
14    "title": "Mechanical Keyboard",
15    "description": "RGB gaming keyboard",
16    "price": 89.99,
17    "inventoryQuantity": 45,
18    "imageUrl": "keyboard.jpg",
19    "category": "Electronics",
20    "discountedPrice": 80.99
21  },
22  {
23    "id": 5,
24    "title": "USB-C Hub",
25    "description": "Multiport USB-C hub",
26    "price": 59.99,
27    "inventoryQuantity": 60,
28    "imageUrl": "hub.jpg",
29    "category": "Electronics"
30  }
31 ]
```

9.7 Update Product (PUT)

The screenshot displays a REST client interface for the SmartShop Product Management API. The active tab is 'PUT Update Product' with the URL `{{baseUrl}}/api/products/1`. The request body is a JSON object representing a product update:

```
1 {
2   "title": "Wireless Mouse",
3   "description": "Ergonomic wireless mouse - updated",
4   "price": 24.99,
5   "inventoryQuantity": 100,
6   "imageUrl": "mouse.jpg",
7   "category": "Electronics"
8 }
```

The response status is **200 OK** with a response time of 30 ms and a body size of 358 B. The response body is shown in JSON format:

```
1 {
2   "id": 1,
3   "title": "Wireless Mouse",
4   "description": "Ergonomic wireless mouse - updated",
5   "price": 24.99,
6   "inventoryQuantity": 100,
7   "imageUrl": "mouse.jpg",
8   "category": "Electronics",
9   "discountedPrice": 22.49
10 }
```

The interface includes various tabs for request details (Body, Cookies, Headers, Test Results) and a bottom bar with options like Connect Git, Console, Terminal, and Globals.

9.8 Delete Product (DELETE)

The screenshot displays a REST client interface for the SmartShop Product API. The top navigation bar includes a search bar, an 'Invite' button, an 'Upgrade' button, and a notification bell. The main workspace shows a list of API endpoints, with 'DEL Delete Product' selected. The interface is divided into several sections:

- Request Section:** The method is set to 'DELETE' and the URL is `{{baseUrl}}/api/products/11`. A 'Send' button is located to the right of the URL.
- Params Section:** A table for query parameters is shown, with columns for 'Key', 'Value', 'Description', and 'Bulk Edit'. The table is currently empty.
- Response Section:** The status is '204 No Content', with a response time of '27 ms' and a size of '112 B'. The 'Body' tab is selected, showing a 'Raw' view of the response.
- Footer:** A status bar at the bottom shows 'Connect Git', 'Console', 'Terminal', and 'Globals'.

Key	Value	Description	Bulk Edit
Key	Value	Description	

204 No Content · 27 ms · 112 B · Save Response

Raw · Preview · Visualize

1

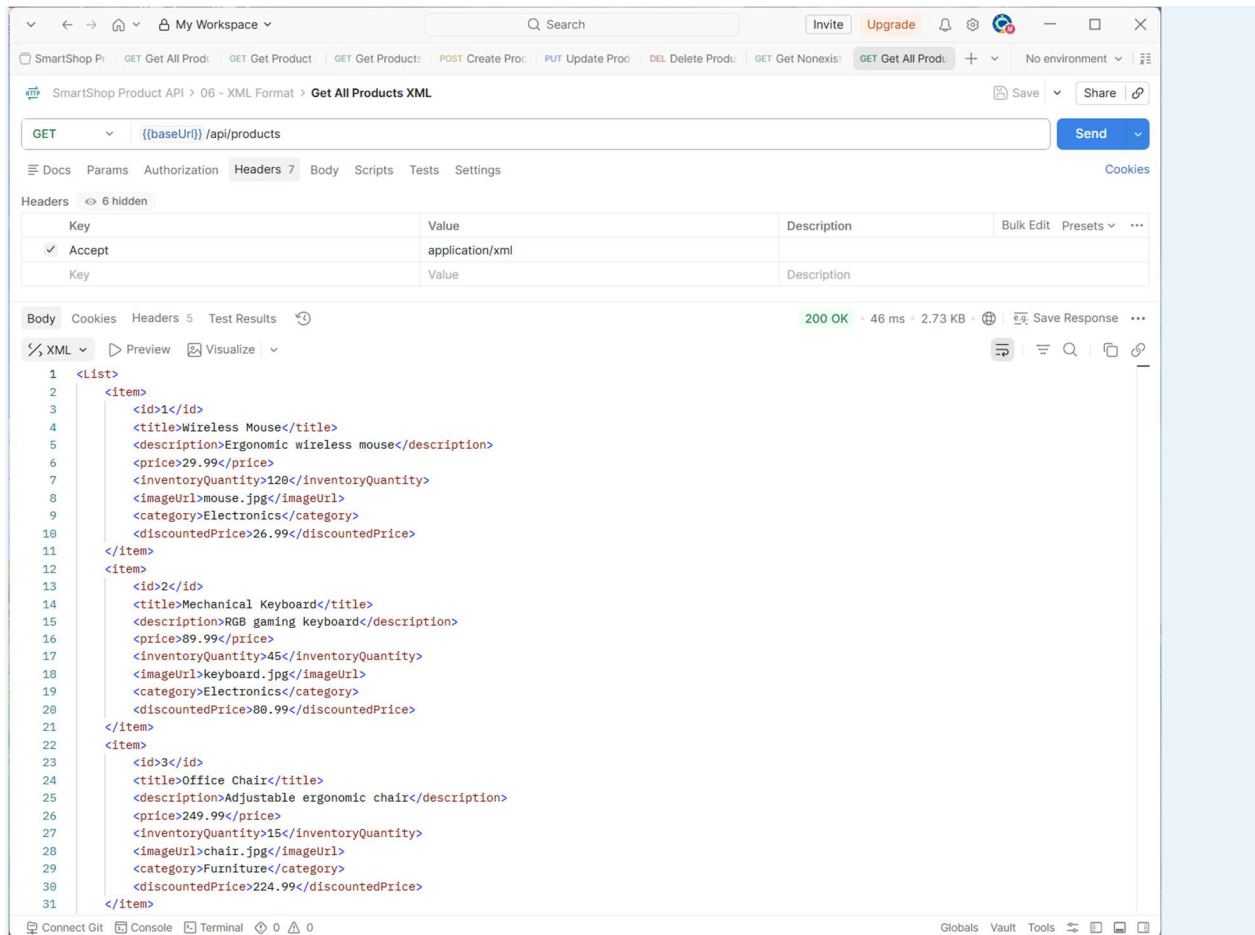
9.9 JSON Response Format

The screenshot displays a REST client interface for the SmartShop Product API. The request is a GET to `{{baseUrl}}/api/products`, which has been successfully executed, returning a 200 OK status with a response time of 17 ms and a body size of 1.9 KB.

The response is in JSON format and contains an array of three product objects. The JSON is as follows:

```
1 [
2   {
3     "id": 1,
4     "title": "Wireless Mouse",
5     "description": "Ergonomic wireless mouse",
6     "price": 29.99,
7     "inventoryQuantity": 120,
8     "imageUrl": "mouse.jpg",
9     "category": "Electronics",
10    "discountedPrice": 26.99
11  },
12  {
13    "id": 2,
14    "title": "Mechanical Keyboard",
15    "description": "RGB gaming keyboard",
16    "price": 89.99,
17    "inventoryQuantity": 45,
18    "imageUrl": "keyboard.jpg",
19    "category": "Electronics",
20    "discountedPrice": 80.99
21  },
22  {
23    "id": 3,
24    "title": "Office Chair",
25    "description": "Adjustable ergonomic chair",
26    "price": 249.99,
27    "inventoryQuantity": 15,
28    "imageUrl": "chair.jpg",
29  }
30 ]
```

9.10 XML Response Format



The XML response format was tested in Postman by sending a request with the `Accept` header set to `application/xml`. The API returned product data as XML, confirming that the REST endpoints support XML content negotiation in addition to JSON.

9.11 Error Handling Examples

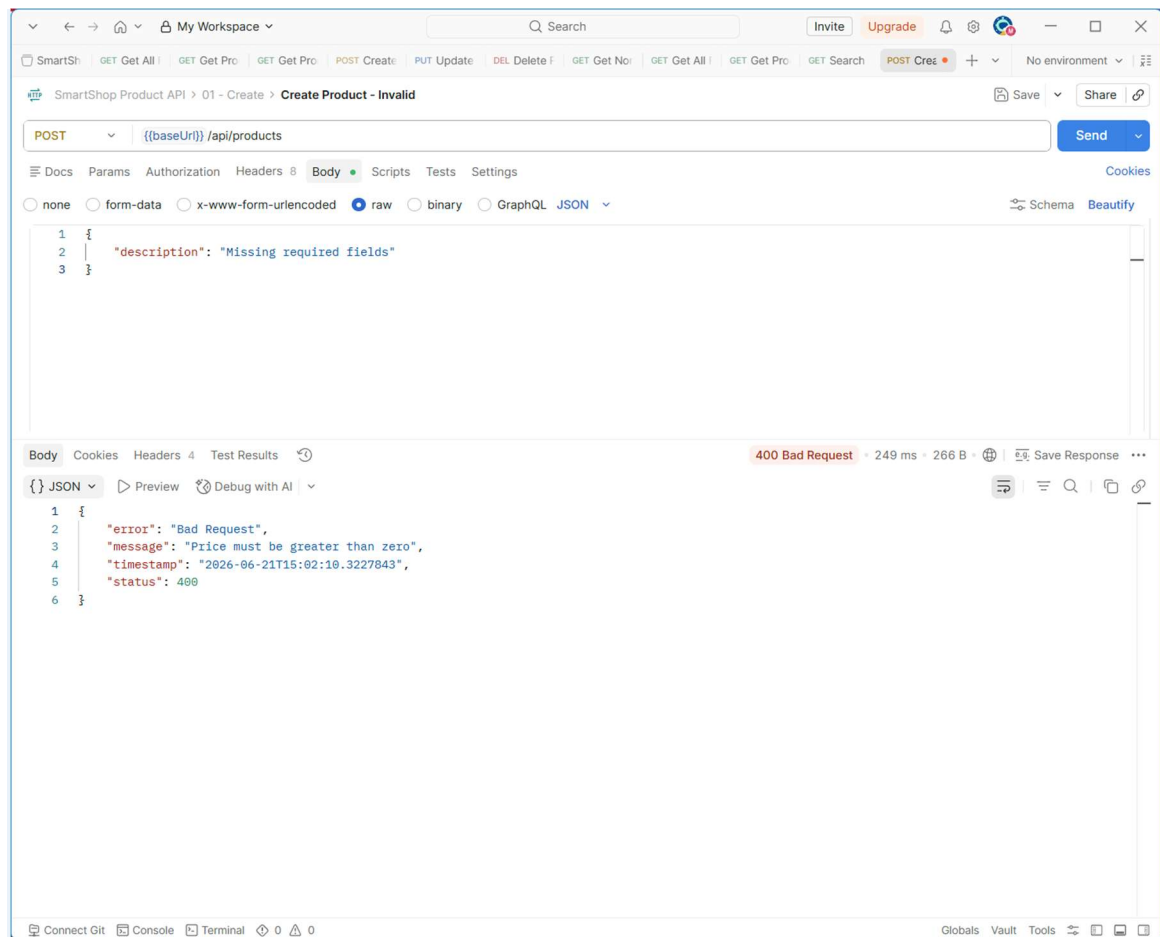
The screenshot displays a REST client interface with the following details:

- Workspace:** My Workspace
- API List:** SmartShop Product API > 05 - Error Handling > Get Nonexistent Product
- Method:** GET
- URL:** `{{baseUrl}}/api/products/9999`
- Response Status:** 404 Not Found (18 ms, 291 B)
- Response Body (JSON):**

```
1 {
2   "error": "Not Found",
3   "message": "Product not found with id: 9999",
4   "timestamp": "2026-06-21T14:22:19.8933356",
5   "status": 404
6 }
```

The interface also includes tabs for Params, Authorization, Headers, Body, Scripts, Tests, and Settings. The Body tab is currently selected, showing the JSON response. The Params tab shows a table with columns Key, Value, and Description.

Key	Value	Description
Key	Value	Description



10. Conclusion

The SmartShop Product Management API provides a backend service for managing product catalog data through RESTful endpoints. The application supports product creation, retrieval, searching, updating, and deletion, with data persisted in a MySQL database through Spring Data JPA and Hibernate. It also returns a calculated discounted price in API responses without storing that value in the database; it supports both JSON and XML response formats, and it provides meaningful HTTP status codes for successful and error scenarios.

11. AI Transparency and Acknowledgment

McGill AI Policy - 3. Transparency and Acknowledgment:

This assignment used AI assistance (Claude Sonnet 4.6, Anthropic) for: learning aid (understanding Spring Boot internals, JPA/Hibernate behavior, Java 17 dependencies, and REST design concepts); code debugging during development; and assembling the technical report document skeleton.